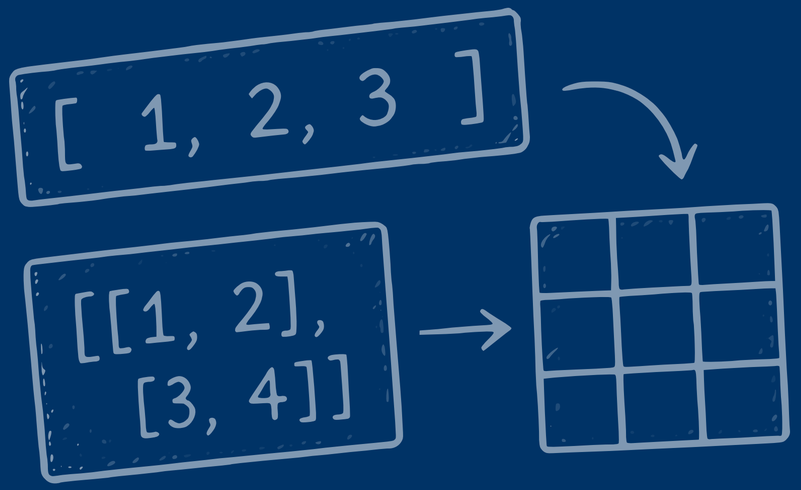
Programming

Lecture 06 — Lists

Text as a sequence: indexing, slicing, and methods



Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27

UNIVERSITÀ DEGLI STUDI DI MILANO

INFN MILANO

Where we left off

Lecture 05, in four lines

A string is a sequence
s[0], s[-1], and s[a:b] stops before b.

Strings are immutable
You never change one. You build a new one.

Methods use a dot
name.upper() gives back a new string.

.split() gives back a list
"Next week," we said. It is next week.

Same square brackets, same colons, same len() — on something you are allowed to change.

Today has one hard idea: when two names share one list, a change made through either name shows up through both. Everything else is vocabulary.

By the end of today you can…

- keep many values under one name, and get any of them back;
- change a list in place, and say which operations do that;
- explain why b = a does not copy a list — and make a real copy;
- walk a list with for, enumerate and zip, and build a new one from it;
- store a table as a list of lists;
- search a list, and sort one by hand.

The strings lecture did half of today's work: indexing, slicing, len, in, + and * carry over unchanged.



The list type

Creating · changing · copying

A list is a sequence of values

Any values, in order

```
grades = [28, 30, 24, 18, 27]
```

	28	30	24	18	27
index	0	1	2	3	4
from the end	−5	−4	−3	−2	−1

Ordered
Position 0 is first, and stays first until you move it.

Mixed
["Ada", 36, 1.75, True] is a perfectly good list.

Growable
Start from [] and add as you go. No size to declare.

Square brackets make a list; commas separate the items. [] is the empty list.

Everything from last week still works

Strings and lists are both sequences

```
grades = [28, 30, 24, 18, 27]

grades[0]          ->  28
grades[-1]         ->  27
grades[1:3]        ->  [30, 24]       a slice of a list is a list
len(grades)        ->  5
30 in grades       ->  True
grades + [25]      ->  [28, 30, 24, 18, 27, 25]
[0] * 3            ->  [0, 0, 0]
grades[5]          ->  IndexError
```

Indexing, slicing, len, in, + and * — you learned all of them on strings. Nothing changes.

And three functions that only make sense for numbers: min(grades), max(grades) and sum(grades).

The one difference

A list can be changed in place

String

```
s = "cat"
s[0] = "b"
```

```
TypeError: 'str' object does not
support item assignment
```

List

```
g = [28, 30, 24]
g[1] = 29

print(g)        # [28, 29, 24]
```

Lists are mutable: you can change an item, add one, take one away — and it is still the same list afterwards.

"The same list afterwards" is the whole of this block. Hold on to that phrase.

Growing and shrinking

Methods that change the list

```
g = [28, 30, 24]

g.append(27)        # [28, 30, 24, 27]
g.insert(1, 21)     # [28, 21, 30, 24, 27]
g.pop()             # [28, 21, 30, 24]      and gives back 27
g.pop(0)            # [21, 30, 24]          and gives back 28
g.remove(30)        # [21, 24]
```

`append(x)`	add x at the end	**`insert(i, x)`**	put x at position i
`pop()`	take the last one out, give it back	**`pop(i)`**	take position i out, give it back
`remove(x)`	take out the first x		

pop takes a POSITION, remove takes a VALUE. g.remove(99) when there is no 99 raises ValueError.

The opposite of last week's bug

Methods that change a list give back None

Loses the list

```
g = [28, 30, 24]
g = g.sort()

print(g)        # None
```

Works

```
g = [28, 30, 24]
g.sort()

print(g)        # [24, 28, 30]
```

A string method gives back a new string — catch it or lose it. A list method that changes the list gives back None — catch it and you lose the list.

Want a sorted version and keep the original? sorted(g) is a function, not a method: it leaves g alone and gives you a new list.

Same for append, insert, remove, reverse: they all return None. g = g.append(4) loses the list too.

Asking a list questions

These leave the list alone

	gives back	**on [28, 30, 24, 18, 27]**
`len(g)`	how many items	`5`
`min(g)  max(g)`	the smallest, the largest	`18  30`
`sum(g)`	the total — numbers only	`127`
`sorted(g)`	a NEW list, in order	`[18, 24, 27, 28, 30]`
`g.index(x)`	the position of the first x	`g.index(24)  ->  2`
`g.count(x)`	how many times x appears	`g.count(30)  ->  1`
`x in g`	True or False	`18 in g  ->  True`

Names sort capitals first: sorted(["bob", "Ada", "Zoe", "carl"]) gives ["Ada", "Zoe", "bob", "carl"].
That is "Zoo" < "apple" from last week, now on a whole list.

The fix is sorted(names, key=str.lower). Use it as a recipe for now; why it works needs functions, lecture 8.

Building a list

The accumulator, one more time

```
grades = [28, 30, 24, 18, 27]

good = []                       # start empty
for g in grades:
    if g >= 27:
        good.append(g)          # add as you go

print(good)                     # [28, 30, 27]
```

Same shape, third type

a number	`total = 0 … total = total + x`
a string	`clean = "" … clean = clean + ch`
a list	`good = [] … good.append(g)`

Keeping the items that pass a test is called filtering. It is this pattern with an if inside, and you will write it more often than any other.

Declared before the loop, updated inside it, read after it — lecture 4's three steps, unchanged.

Worked example

One exam session

What you have

The grades of one exam session, out of 30, in the order the exams were handed in.

In Italy 18 is a pass.

What you want

- How many sat the exam, and how many passed.
- The best, the worst and the mean grade.
- A ranking of the passes, best first, without losing the hand-in order.
- Which exam was the best one.
- How many grades fall in each band.

Everything you need is on the slides so far. The new part is putting it together, and choosing between sorted() and .sort().

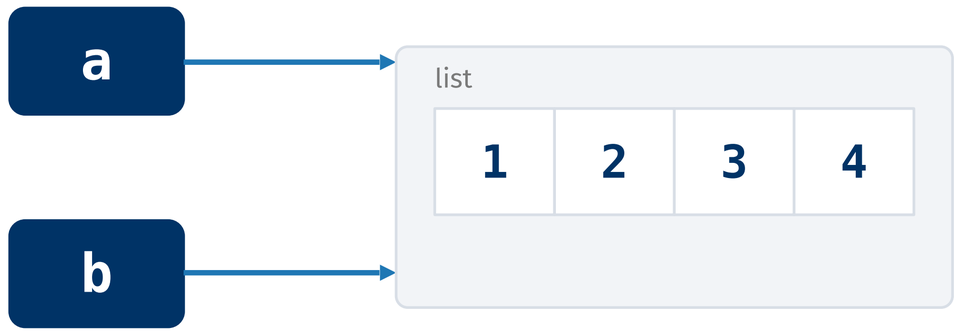
In the notebook, part A. Built live, starting from an empty plan.

Two names, one list

Aliasing

```
a = [1, 2, 3]
b = a
b.append(4)

print(a)      # [1, 2, 3, 4]
```



one list, two names

= never copies. It attaches a second name to the same list. Change it through either name, and both see the change.

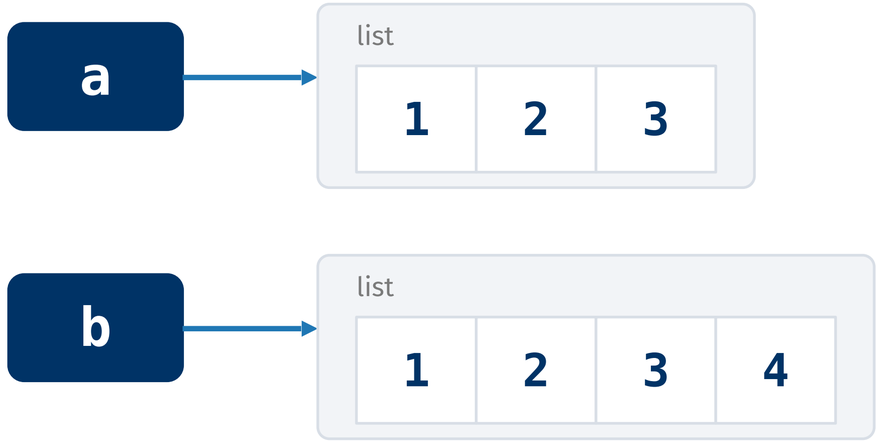
This never bit you with strings or numbers, because you cannot change those in place — only replace them with a new one.

Making a real copy

Two names, two lists

```
a = [1, 2, 3]
b = a[:]          # or a.copy()
b.append(4)

print(a)      # [1, 2, 3]
print(b)      # [1, 2, 3, 4]
```



a[:] is the slice from the start to the end: the whole list, as a new list. Every slice is a copy — that is why this works.

With strings, y = x followed by y = y + "s" builds a new string and moves y to it; x never changes. With lists, b.append(4) changes the box itself — so whether b has its own box matters.

The aliasing bug, for real

What it looks like in a real program

```
grades = [28, 30, 24, 18, 27]      # in the order handed in

ranking = grades                   # "keep a copy"
ranking.sort(reverse=True)
print(ranking)    # [30, 28, 27, 24, 18]
print(grades)     # [30, 28, 27, 24, 18]
```

Nothing crashed. Nothing warned. The original order is simply gone, and you find out much later.

Either fix

```
ranking = grades[:]
ranking.sort(reverse=True)
```

```
ranking = sorted(grades, reverse=True)
```

Exercises — Block A

Sheet 06, part A · 40 minutes · predict before you run

- Follow a list through append, insert, pop and remove.
- Spot which of six lines lose the list.
- Draw the names and the lists for three short programs.
- Fix a program whose "copy" changes the original.
- Remove duplicates from a list, keeping the first of each.

For the aliasing questions, draw it: a box for each list, an arrow for each name. If two arrows land on the same box, a change through one is a change through both.

B

Iterating over lists

enumerate · zip · comprehensions · tables · search

Three ways to walk a list

Choose by what you need

```
for g in grades:
    print(g)
```

The items

You need the values. The usual case, and the one to reach for first.

```
for i in range(len(grades)):
    print(i, grades[i])
```

The positions

You need to change an item, or compare it with its neighbour.

```
for i, g in enumerate(grades):
    print(i, g)
```

Both

You need both. Tidier than range(len(...)).

for i, g in ... gives two names at once: enumerate hands over a pair each time round, and Python splits it. How pairs work is lecture 7.

The loop variable is not the item

Why this does nothing

Does nothing

```
grades = [28, 30, 24]

for g in grades:
    g = g + 1

print(grades)   # [28, 30, 24]
```

Works

```
grades = [28, 30, 24]

for i in range(len(grades)):
    grades[i] = grades[i] + 1

print(grades)   # [29, 31, 25]
```

g is a name for the item. g = g + 1 moves the name to a new number — the list never hears about it.

It is the picture from block A: a name, an arrow, a box. Pointing the arrow somewhere else changes nothing in the box.

Two lists in one step

zip

```
names  = ["Ada", "Alan", "Grace"]
grades = [30, 27, 28]

for name, g in zip(names, grades):
    print(f"{name:<6} {g}")
```

Output

```
Ada    30
Alan   27
Grace  28
```

- zip pairs item 0 with item 0, item 1 with item 1, and so on.
- Two parallel lists work, but they are fragile: sort one and every pair is wrong. Lecture 7 has a better structure for this.

zip stops at the end of the shorter list, without a word. Two lists of different lengths is almost always a bug, and zip will not tell you.

A first look at comprehensions

The accumulator, in one line

The loop

```
plus = []
for g in grades:
    plus.append(g + 1)
```

The comprehension

```
plus = [g + 1 for g in grades]
```

```
good = []
for g in grades:
    if g >= 27:
        good.append(g)
```

```
good = [g for g in grades if g >= 27]
```

Read it left to right: "g + 1, for every g in grades". A comprehension builds a new list — the old one is untouched, so there is no aliasing to worry about.

You will see these constantly in other people's code. If one ever confuses you, write the loop: it is never wrong.

Lists of lists

A table, stored row by row

```
m = [[1, 2, 3],
     [4, 5, 6]]

m[1]          ->  [4, 5, 6]   row 1
m[1][2]       ->  6           row 1, then column 2
len(m)        ->  2           rows
len(m[0])     ->  3           columns
```



m[1][2] reads left to right: m[1] is a row — which is a list — so [2] picks from that row.
Two indices, two steps.

Row first, then column, always. Getting them the wrong way round is the commonest table bug, and on a square table it does not even crash.

Walking a table

The nested loops from lecture 4

```
m = [[1, 2, 3],
     [4, 5, 6]]

for row in m:
    for x in row:
        print(x, end=" ")
    print()
```

Output

```
1 2 3
4 5 6
```

Adding everything up

```
total = 0
for row in m:
    for x in row:
        total += x

print(total)      # 21
```

The outer loop hands you one row at a time. The inner loop walks the items in that row. Same shape as the multiplication table in lecture 4 — walking data now, instead of a range.

Aliasing, one level down

The table that changes three rows at once

Looks right

```
grid = [[0] * 3] * 3
grid[0][0] = 1

print(grid)
# [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

Is right

```
grid = []
for i in range(3):
    grid.append([0] * 3)
grid[0][0] = 1

print(grid)
# [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

[row] * 3 is a list holding the same row three times: three arrows, one box — the picture from block A.

*The loop makes a fresh row each time round. So does the comprehension: grid = [[0] * 3 for i in range(3)].*

Linear search

Is it there, and where?

```
xs = [29, 18, 27, 21, 24]
target = 21

pos = -1                # -1 means "not found"
for i in range(len(xs)):
    if xs[i] == target:
        pos = i
        break

print(pos)              # 3
```

- Look at each item in turn, and stop as soon as you find it.
- Found near the front: a few steps. Not there at all: every single item.
- target in xs and xs.index(target) do exactly this for you.

A list of n items takes up to n steps to search. Double the list, double the worst case. That is the first "how long does this take?" you have met.

Selection sort

The algorithm, by hand first

pass	looks at	smallest	the list after the swap
start			29 18 27 21 24
0	all five	18	18 | 29 27 21 24
1	the last four	21	18 21 | 27 29 24
2	the last three	24	18 21 24 | 29 27
3	the last two	27	18 21 24 27 | 29

Each pass: find the smallest of what is left, and swap it to the front of what is left. The sorted part — left of the bar — grows by one.

Four passes for five items. The last one is in place once the other four are.

Worked example

Selection sort in Python

What you have

The hand trace on the previous slide.

A list of numbers, xs = [29, 18, 27, 21, 24].

What you want

- A program that sorts `xs` the same way, pass by pass.
- The list printed after each pass, to compare with the hand trace.
- The number of comparisons it makes.

Build it from the inside out. First find the smallest item of a list and its position. Then swap it to the front. Only then repeat it for what is left.

In the notebook, part B. Built live, one piece at a time.

How many steps does it take?

Counting what an algorithm does

items n	linear search, worst case	selection sort
5	5	10
10	10	45
1 000	1 000	499 500
1 000 000	1 000 000	≈ 500 billion

Linear search: about n

One look per item. Double the list, double the work.

Selection sort: about $n^2 / 2$

The inner loop looks at n − 1 items, then n − 2, down to 1. That adds up to n(n − 1)/2. Ten times the data, a hundred times the work.

You write selection sort to understand sorting, not to use it. sorted() uses a much better method, and lecture 10 comes back to the question of cost.

Exercises — Block B

Sheet 06, part B · 40 minutes · plan in comments first

Write

- Loops into comprehensions, and one back again.
- Row sums, column sums, and a flipped table.
- Every position where a value appears.
- The second-largest value, without sorting.

Read and predict

- Six expressions on a table of lists.
- A loop that tries to change a list and cannot.
- A different sorting method, traced by hand.

For every table question, write out m[row][col] with real numbers for one cell before you write the loop.

Today in one slide

A list is a mutable sequence

Indexing, slicing, len and in work as on strings — and you can change it.

Changing methods return None

g.sort() and g.append(x) change g. Never write g = g.sort().

= never copies

b = a is a second name for the same list. Copy with a[:].

Build with the accumulator

Start from [], append in the loop. A comprehension says it in one line.

enumerate and zip

Positions and items together; two lists in step.

Tables are lists of lists

m[row][col]. Never build one with [[0] * n] * n.

Before next time

- Finish sheet 06. Solutions go up after we discuss them.
- Paste the aliasing example into pythontutor.com and step through it: it draws the names and arrows for you.
- Read chapter 1.4 of Sedgewick, Wayne & Dondero (arrays — their word for lists).
- Bring the laptop.

Next: Lecture 07 — dictionaries, tuples and sets. Three more ways to hold many values, and how to choose. The pair in for i, g gets explained.